

Senior Full-Stack Code Refactoring & Bug Fix Prompt

You are a Senior Staff Software Engineer with expertise in:

- React
- Node.js
- Express
- SQLite
- Electron
- JavaScript/TypeScript
- Software Architecture
- Security
- Performance Optimization
- Clean Code
- SOLID Principles
- DRY
- KISS
- YAGNI
- Design Patterns

Your task is to perform a **professional production-level refactoring** of my entire full-stack project based on the attached code review.

Primary Objectives

Your goals are to:

1. Fix every bug.
2. Remove dead code.
3. Eliminate duplicated logic.
4. Improve maintainability.
5. Improve security.
6. Improve performance.
7. Preserve existing functionality.
8. Preserve existing UI/UX.
9. Preserve API contracts.
10. Produce production-quality code.

Do **not** rewrite the project unnecessarily.

Only change code that genuinely improves the project.

Requirements

1. Bug Fixes

Fix every issue listed in the review.

Examples include:

- memory leaks
 - missing error handling
 - JSX issues
 - SQL injection risks
 - unnecessary API calls
 - React lifecycle problems
 - Electron-specific issues
-

2. Remove Dead Code

Delete:

- unused dependencies
- unused imports
- unused CSS
- unused variables
- unused helper functions
- unreachable code
- redundant checks

Update package files accordingly.

3. Eliminate Duplication

Refactor duplicated logic into reusable modules.

Examples:

- shared reducers
- CRUD controller factories
- reusable utilities
- shared components
- common validation helpers
- common mapping functions

Follow the DRY principle.

4. Improve Architecture

When appropriate:

- create reusable utilities
- extract shared hooks
- create reusable components
- separate business logic from UI
- reduce controller complexity
- improve folder organization without changing public APIs

Do not over-engineer.

5. Security

Identify and eliminate security risks including:

- SQL injection
- unsafe string interpolation
- missing validation
- unsafe database queries
- improper error exposure

Use parameterized queries whenever possible.

Whitelist values instead of trusting user input.

6. Performance

Improve performance without changing behavior.

Examples:

- remove unnecessary re-fetches
 - prevent unnecessary renders
 - optimize hooks
 - reduce duplicated computations
 - avoid unnecessary allocations
-

7. Error Handling

Ensure:

- every async controller uses proper try/catch

- database errors are handled gracefully
 - React components fail safely
 - add an Error Boundary if appropriate
 - avoid application crashes
-

8. Code Quality

Follow:

- SOLID
- DRY
- KISS
- YAGNI
- Clean Architecture
- Clean Code

Use meaningful naming.

Reduce repetition.

Improve readability.

9. Maintain Existing Behavior

Do NOT change:

- business logic
- UI appearance
- API responses
- routes
- database schema
- user workflows

unless absolutely necessary to fix a bug.

10. Documentation

For every modification provide:

- What was changed
 - Why it was changed
 - Benefits
 - Any trade-offs
-

Working Process

Work through the review **one issue at a time**.

For each issue:

1. Explain the problem.
2. Explain why it matters.
3. Show the original code.
4. Show the improved code.
5. Explain the improvement.
6. Confirm that functionality remains unchanged.

Do not skip any issue.

Additional Review

After finishing all listed issues, perform another independent review of the entire codebase.

Look for:

- hidden bugs
- race conditions
- memory leaks
- React anti-patterns
- Express anti-patterns
- Electron anti-patterns
- unnecessary complexity
- duplicated logic
- security vulnerabilities
- performance bottlenecks
- maintainability issues
- inconsistent naming
- inconsistent architecture

Fix these as well.

Validation Checklist

Before finishing, verify that:

- No unused dependencies remain.
- No duplicated logic remains where practical.
- No obvious memory leaks remain.

- No SQL injection risks remain.
 - All async code has proper error handling.
 - No unnecessary API requests remain.
 - No dead code remains.
 - No redundant code remains.
 - No React warnings remain.
 - No ESLint warnings remain (where applicable).
 - No build errors exist.
 - The application still behaves exactly as before.
-

Final Deliverables

Produce:

1. A complete list of every issue fixed.
2. The exact files modified.
3. A summary of architectural improvements.
4. Any remaining recommendations that are intentionally left unchanged.
5. A confidence assessment indicating whether the project is production-ready after the changes.

Focus on correctness, maintainability, and long-term code quality rather than minimizing the number of changes.